

RAMCO INSTITUTE OF TECHNOLOGY
Department of Computer Science and Engineering
Academic Year: 2019- 2020 (Odd Semester)

Date: 20.08.2019

Degree, Semester & Branch: III Semester B.E. Computer Science and Engineering

Course Code & Title: CS8391 Data Structures

Name of the Faculty member: Mrs.B.Vijayalakshmi, AP(SG)/CSE

Innovative practice: Learn by Doing – Complete the code and trace the execution

Topic: Binary Search Tree & AVL Tree

Objectives of the activity:

- To enable the students to familiarize the concepts of tree construction and improve the programming skills.

Justification for choosing the topic:

Binary search tree and AVL tree is a non-linear data structure in which the construction of these trees plays a vital role in performing search operations effectively. This process and its implementation need to be understood by the students, hence learning by doing activity based method is employed for the construction and tracing the execution flow.

Strategy for Team Formation:

Three members combined to form as a team. The team was a heterogeneous team consisting of one bright student, one slow learner and average student. Totally 16 teams were formed.

Activity:

Complete the code and trace the execution is an interesting activity in which the student has to think and complete the given code and trace its execution for the given data within a given time. It helps them to understand and visualize how the logic for insertion works.

The activity was conducted as a team event in the classroom in Binary search tree and AVL tree. Every team was given with a partial code to perform a specific task. The incomplete code may be of performing insertion, finding minimum and maximum element and tree traversal. They have to complete the code within duration of 15 minutes. Based on the completed code they have to find the position where to insert the given value into the given tree and trace the flow of execution of the block of statements for performing the insertion.

Observations made:

The completed code was corrected during the hour itself and the following observations were made.

- Some of the students have doubt on the insertion, rotation in AVL tree.

- Some need more clarification on the flow of execution.
- Additional examples shall be given them to make understand the concept easily.

Outcome:

This activity was very useful to identify whether the students understand the concept of tree construction and the logic behind it or not. Through this activity, the students are able to analyse the flow of execution clearly.

Relevance Mapping - This activity helps in attaining the following Objectives – PO mapping:

Objectives / PO	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO1 0	PO1 1	PO1 2
O1	3	3	3	2	-	-	-	-	3	3	-	1

The figure displays two examples of student work on a Binary Search Tree (BST) problem. Both students are from Ramco Institute of Technology, Department of Computer Science and Engineering, Academic Year 2019-2020 (IInd Semester). The problem asks to complete the code and trace the execution for inserting an element into a BST.

Left Student (Vijayalakshmi):

- Code:**

```

1 //
2
3 #include <iostream>
4 using namespace std;
5
6 struct tnode {
7     int data;
8     tnode *left;
9     tnode *right;
10 };
11
12 tnode* insert(tnode* root, int x) {
13     if (root == NULL) {
14         root = new tnode;
15         root->data = x;
16         root->left = NULL;
17         root->right = NULL;
18         return root;
19     }
20     if (x < root->data) {
21         root->left = insert(root->left, x);
22     }
23     else if (x > root->data) {
24         root->right = insert(root->right, x);
25     }
26     return root;
27 }
28
29 void preorder(tnode* root) {
30     if (root == NULL) {
31         return;
32     }
33     cout << root->data << " ";
34     preorder(root->left);
35     preorder(root->right);
36 }

```
- Tree Diagram:** A BST with root 27. Left child is 10, right child is 329. Node 10 has a left child 100. Node 100 has a right child 105 (circled in blue).

Right Student (Vijaya Lakshmi):

- Code:**

```

1 //
2
3 #include <iostream>
4 using namespace std;
5
6 struct tnode {
7     int data;
8     tnode *left;
9     tnode *right;
10 };
11
12 tnode* insert(tnode* root, int x) {
13     if (root == NULL) {
14         root = new tnode;
15         root->data = x;
16         root->left = NULL;
17         root->right = NULL;
18         return root;
19     }
20     if (x < root->data) {
21         root->left = insert(root->left, x);
22     }
23     else if (x > root->data) {
24         root->right = insert(root->right, x);
25     }
26     return root;
27 }
28
29 void preorder(tnode* root) {
30     if (root == NULL) {
31         return;
32     }
33     cout << root->data << " ";
34     preorder(root->left);
35     preorder(root->right);
36 }

```
- Tree Diagram:** A BST with root 10. Left child is 5, right child is 15. Node 5 has a left child 4. Node 4 has a right child 145 (circled in blue).

Figure 1: Partial code for Binary search tree and the answers written by the students

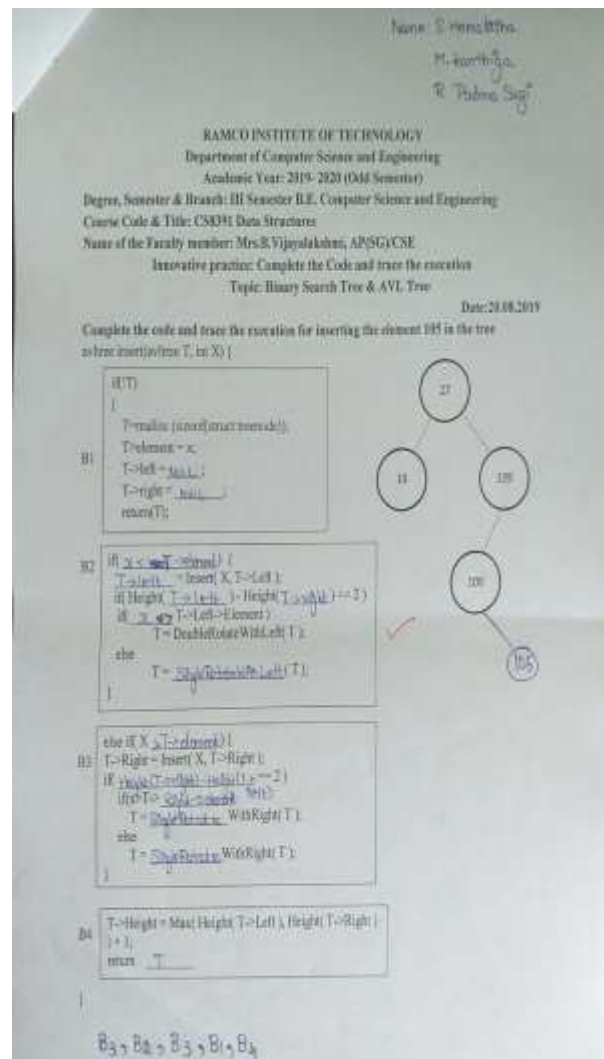
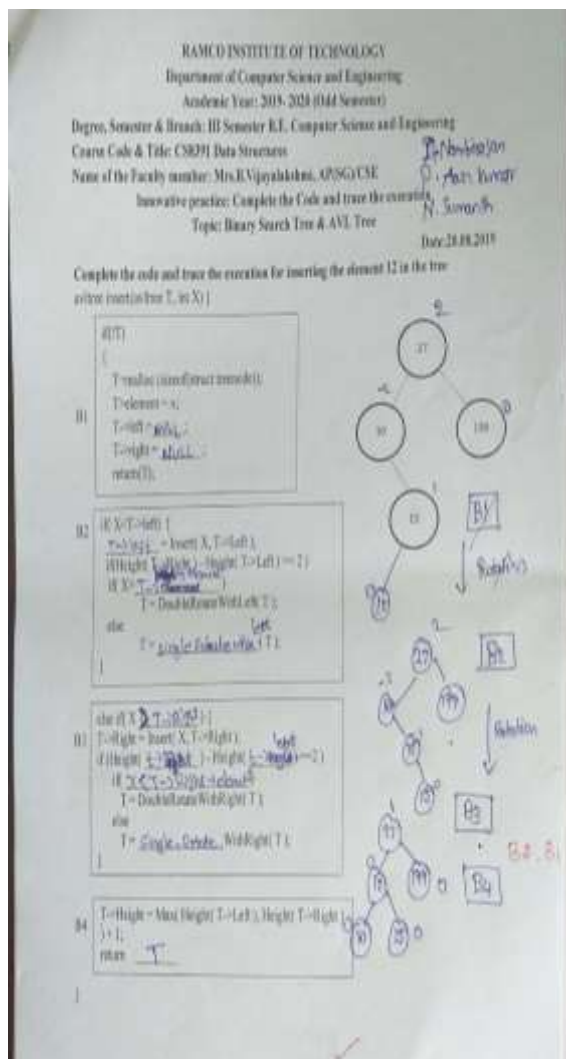


Figure 1: Partial code for AVL tree and the answers written by the students

Reflective Report

Pre-implementation:

Identified Challenges:

- Some of the students feel easy to insert the new element in a tree but they feel difficult for tracing the execution flow.
- Some of the students who were not interested in programming were not show their active participation.

Addressing challenges

- I gave demo to the students for the creation of node and inserting a node in the tree.
- Animations were shown to them to get clarity in the process.
- The students were motivated to have the better coding practice

Post implementation

- Students can understand easily the execution of tree construction.
- It was a different experience for them to learn the inner working of a code which creates interest to them for writing programs.

Prepared By,
Mrs.B.Vijayalakshmi

Approved By
Dr.K.Vijayalakshmi